

Reviewer Pack — SOS Moment Matrix Rank Bounded by Square Root of Variables f...

Entry 00198069e342 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 09:28:49 UTC

SOS Moment Matrix Rank Bounded by Square Root of Variables for Max-CUT

Entry ID: 00198069e342 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 09:28:49 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For any max-CUT instance on n variables, the rank of the degree- d SOS moment matrix is at most $O(\sqrt{n})$ if $d \leq 2$, and strictly increases with d . Any 0.879-approximator for max-CUT must have a moment matrix violating this rank bound for $d=2$.

Rationale (proposer's reasoning):

The SOS hierarchy's moment matrix encodes polynomial identities over the max-CUT feasible region. Real algebraic geometry provides tools to analyze its rank, which reflects the complexity of the relaxation. A low-rank bound for $d=2$ implies structural limitations, while higher d requires increased rank to capture the problem's geometry.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): be6b1fa27f0fea8c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 20 random Max-CUT instances per $n \in \{16, 25, 36\}$ (60 seeds total), solve degree-2 and degree-3 SOS SDPs, compute numerical rank (singular values $> 1e-6 \times \text{top}$). Conjecture is SUPPORTED iff $\geq 80\%$ of $d=2$ instances have rank $\leq 2\lceil\sqrt{n}\rceil$ AND mean $d=3$ rank exceeds mean $d=2$ rank by $\geq \sqrt{n}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Max-CUT SOS hierarchy moment matrix rank bound -sum of squares Lasserre relaxation Max-CUT degree-2 approximation 0.879 - low-rank moment matrix semidefinite relaxation Max-Cut Goemans-Williamson

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_max_cut_instance(n):
    edges = []
    for i in range(n):
        for j in range(i+1, n):
            if random.random() < 0.5:
                edges.append((i, j))
    return edges

def sdp_rank(matrix):
    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for col in range(n):
            pivot_row = -1
            for row in range(rank, m):
                if A[row][col] != 0:
                    pivot_row = row
                    break
            if pivot_row == -1:
                continue
            A[pivot_row], A[rank] = A[rank], A[pivot_row]
            rank += 1
            for row in range(rank, m):
                factor = A[row][col] / A[pivot_row][col]
                for j in range(n):
                    A[row][j] -= factor * A[pivot_row][j]
        return rank

    return gaussian_elimination(matrix)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [16, 25, 36]
    results_d2 = []
    results_d3 = []

    for n in n_values:
        for _ in range(20):
            instance = generate_max_cut_instance(n)
            # Construct degree-2 SOS moment matrix
            P2 = [[0] * (n+1) for _ in range(n+1)]
            for u, v in instance:
                P2[u][v] += 1
                P2[v][u] += 1
                P2[n][u] += 1
                P2[n][v] += 1
            rank2 = sdp_rank(P2)

            # Construct degree-3 SOS moment matrix
            P3 = [[0] * (n+1) for _ in range(n+1)]
            for u, v in instance:
                for w in range(n):
                    if (u, v, w) not in instance and (u, w, v) not in instance and (v, u, w) not in instance:
                        P3[u][v] += 1
                        P3[v][u] += 1
                        P3[w][u] += 1
                        P3[w][v] += 1
            rank3 = sdp_rank(P3)

            results_d2.append(rank2)
            results_d3.append(rank3)

    mean_rank_d2 = sum(results_d2) / len(results_d2)
    mean_rank_d3 = sum(results_d3) / len(results_d3)

    conjecture_holds = (mean_rank_d2 <= 2 * math.ceil(math.sqrt(n_values[0])) and
                        mean_rank_d3 > mean_rank_d2 + math.sqrt(n_values[0]))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Rank",
        "metric_value": (mean_rank_d2, mean_rank_d3),
        "instances_tested": len(results_d2) + len(results_d3),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [i for i in range(2, 50, 2)]

    results_d2_total = []
    results_d3_total = []
    conjecture_holds_count = 0

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")

    if result["conjecture_holds"]:
        conjecture_holds_count += 1
    results_d2_total.extend(result["metric_value"][0] for _ in range(40))
    results_d3_total.extend(result["metric_value"][1] for _ in range(40))

mean_rank_d2 = sum(results_d2_total) / len(results_d2_total)
mean_rank_d3 = sum(results_d3_total) / len(results_d3_total)
support_fraction = conjecture_holds_count / len(seeds)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank_d2:.4f} std={math.sqrt(sum((x - mean_rank_d2)**2)):.4f}")
elif any(result["conjecture_holds"] == False for result in results):
    first_failing_seed = seeds[next(i for i, result in enumerate(results) if not result["conjecture_holds"])]
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_980de3dd.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_980de3dd.py", line 65, in run_trial
    rank2 = sdp_rank(P2)
             ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_980de3dd.py", line 46, in sdp_rank
    return gaussian_elimination(matrix)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_980de3dd.py", line 41, in gaussian_elimination
    factor = A[row][col] / A[pivot_row][col]
              ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError in the gaussian_elimination routine before any rank data was produced, so the pre-registered support condition could not be evaluated.
| next: Replace the hand-rolled Gaussian elimination with numpy.linalg.matrix_rank (or SVD-based numerical rank with the specified 1e-6 tolerance) and rerun the 60 seeds.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	742810
2	propose	ollama_remote	qwen3:8b	0	0	665206
3	preregistration	claude_max	opus	0	0	7453
4	novelty	claude_max	opus	0	0	5010
5	novelty	claude_max	opus	0	0	7717
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14672
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12733
8	judge	claude_max	opus	0	0	4281

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1459882 ms total latency. Provider mix: {'ollama_remote': 4, 'claude_max': 4}

(full prompt+response transcripts available in research/audit/00198069e342.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/00198069e342.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/00198069e342.tar.gz (if generated)